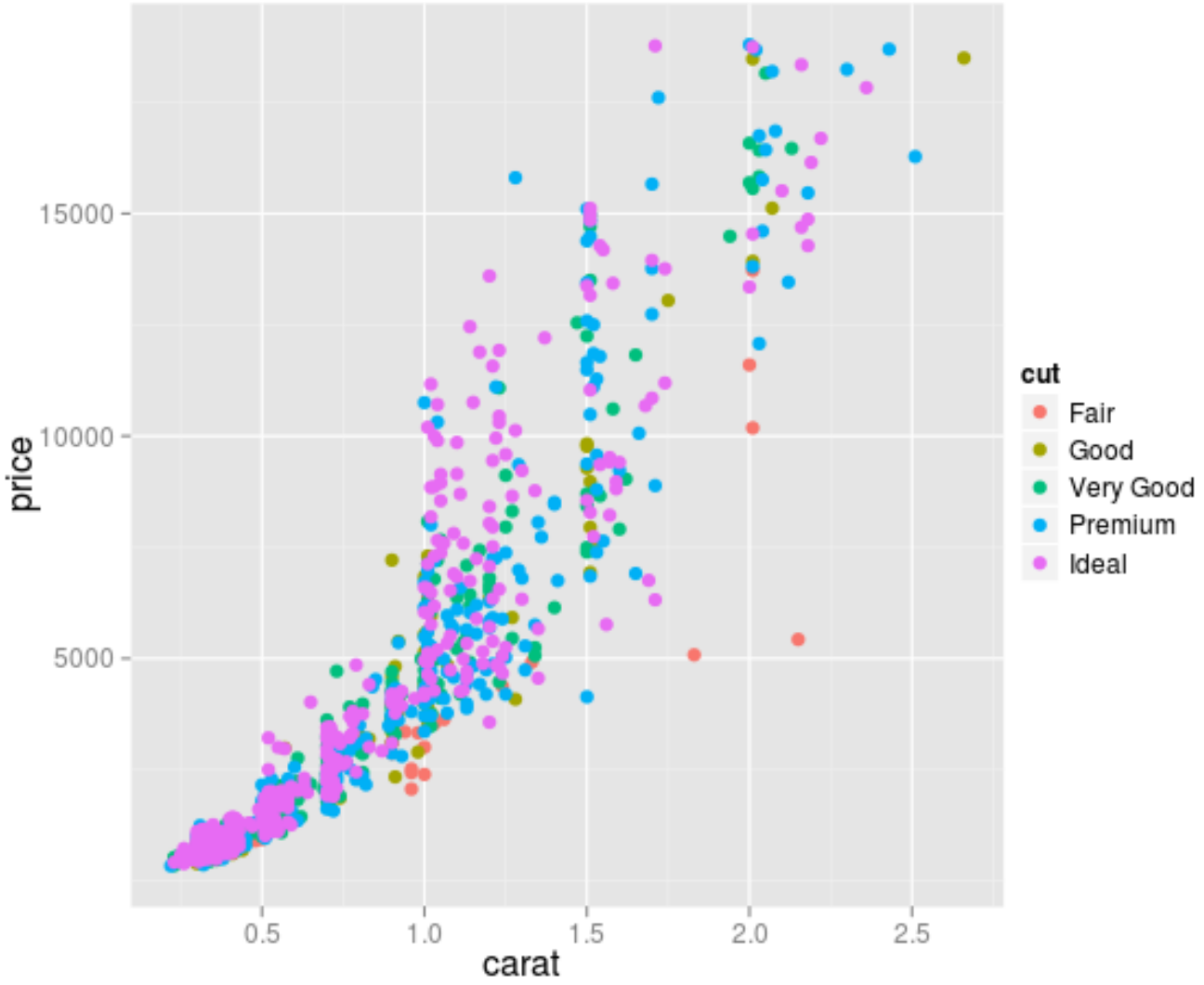
Tree-based Methods

Yu Sheng

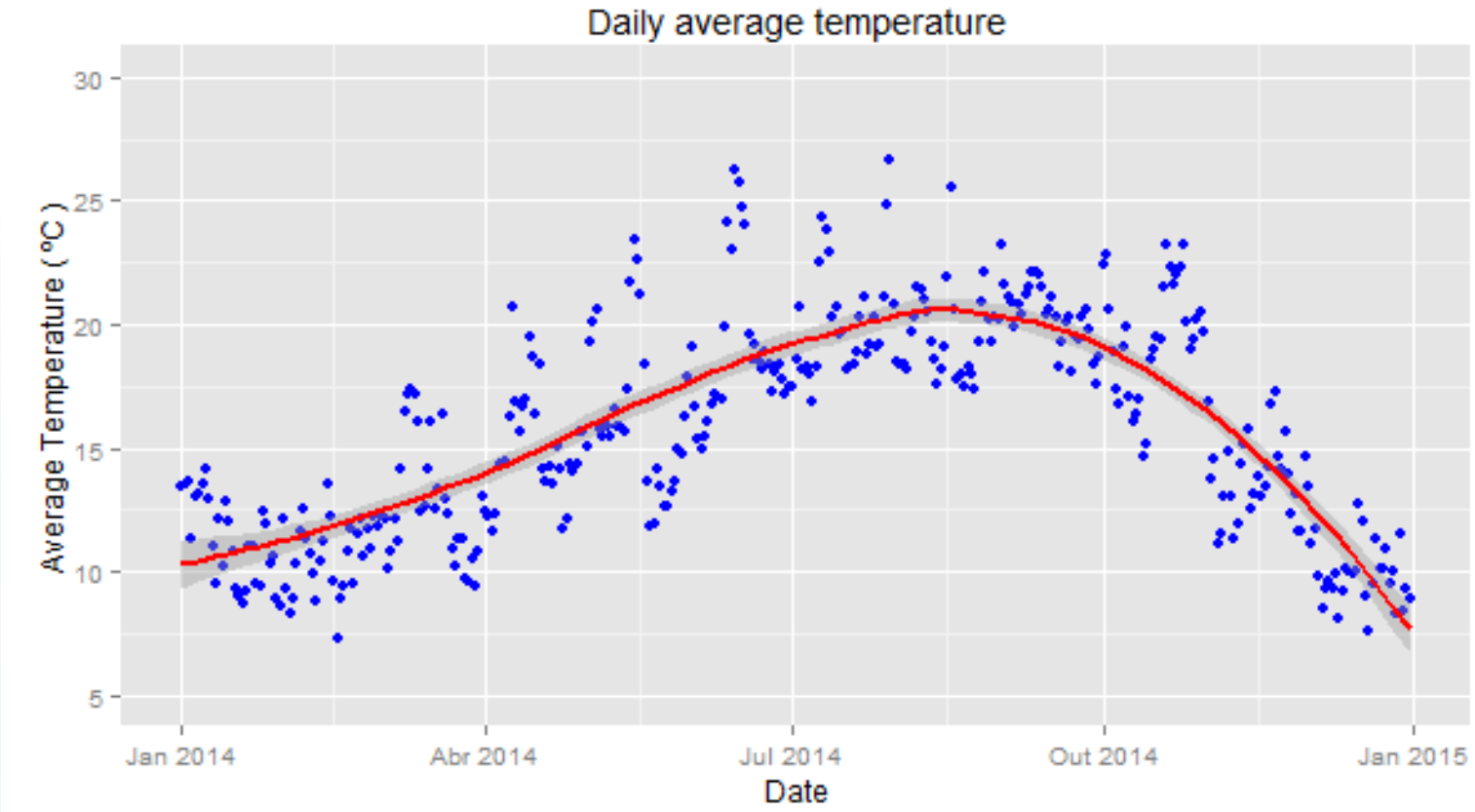
Center for Statistical Science

Decision Trees

- Consider a discrete approximation of a function in a 1-dimensional space.



- Consider a discrete approximation of a function in a 1-dimensional space.



Discrete Approximation

- Approximate by sample mean of the region

$$\hat{f}(x) = \sum_m c_m I\{x \in R_m\}.$$

- For classifications, predict the class with the most frequent one in the region

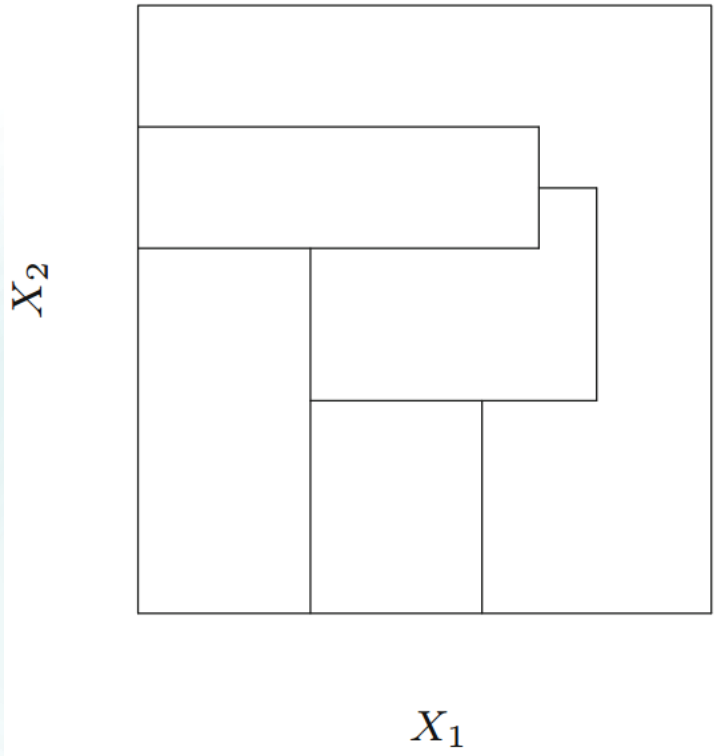
$$k(m) = \operatorname{argmax}_k \sum_{x_i \in R_m} I(y_i = k).$$

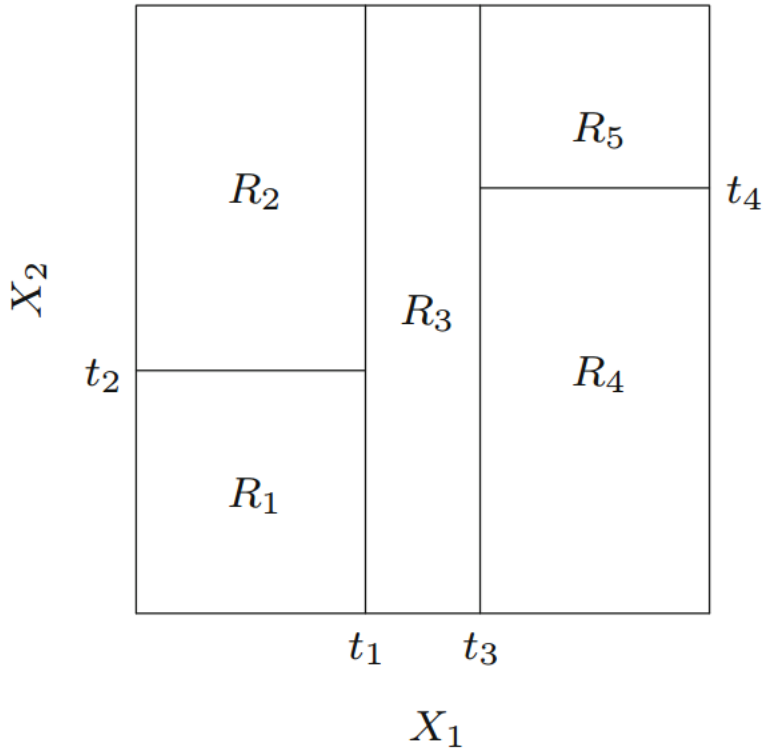
- It is possible to segment the space by a grid in 1 and 2-dimensional, or maybe even 3-dimensional spaces.
- For larger p, it is not practical. – Why?

清华大学统计学研究中心

Discrete Approximation

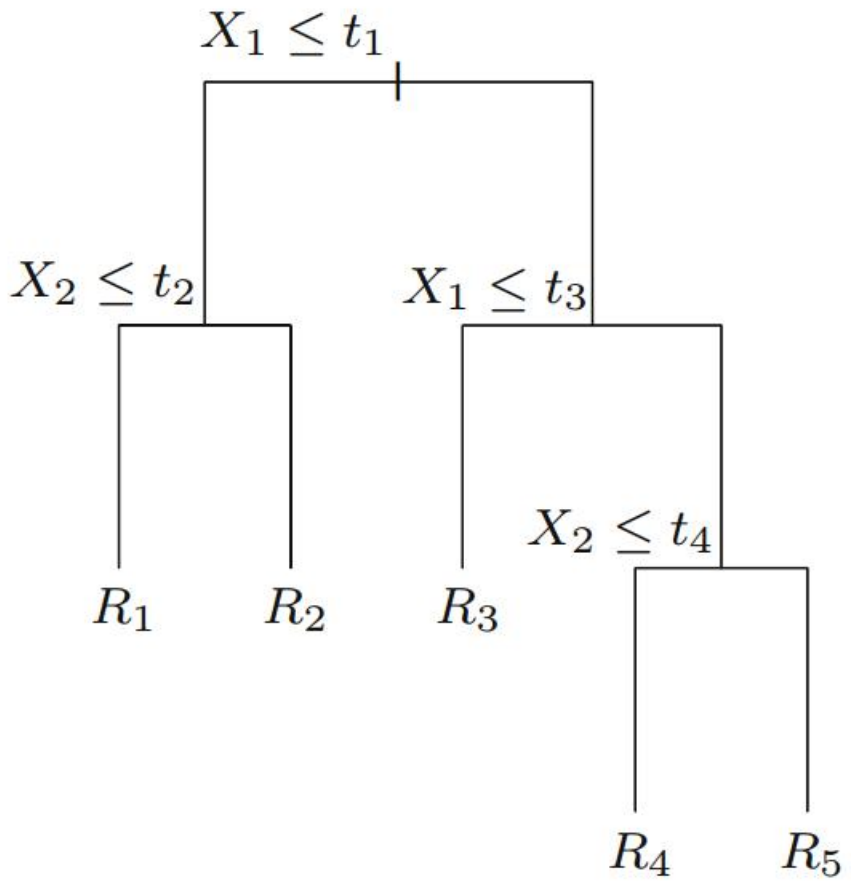
- However, there are smarter ways to segment a feature space.

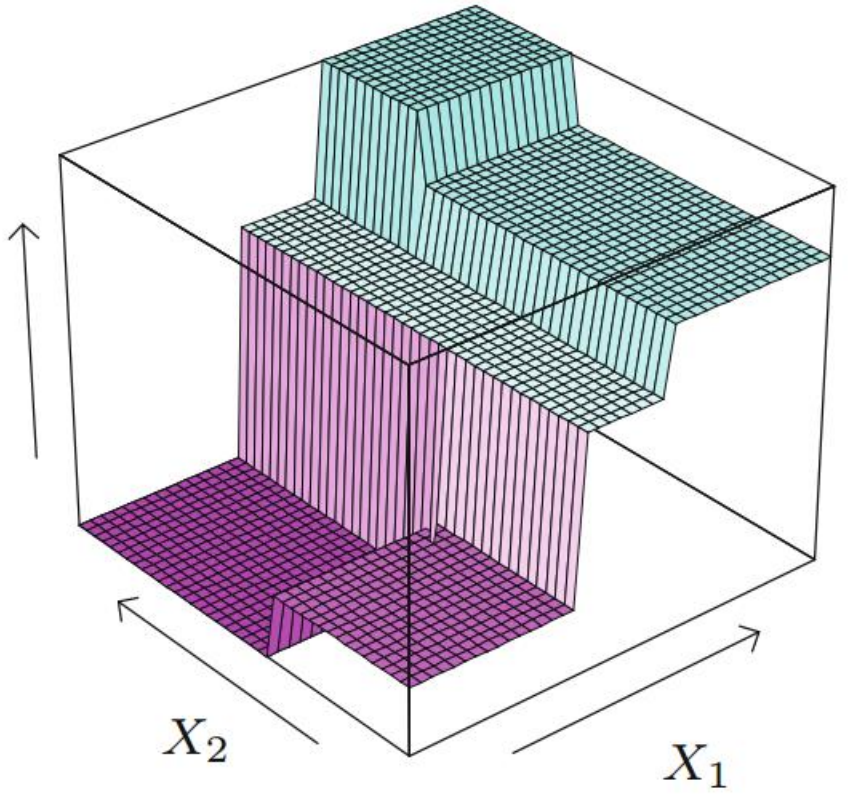




清华大学统计学研究中心

Discrete Approximation





- Example of a decision tree (some nodes are merged):



- We only consider binary trees (don't have to be **complete**):



a complete binary tree

Terminology for Trees

- In keeping with the tree analogy, the regions $R_1, \ldots, R_M$ are known as **terminal nodes**.
- Decision trees are typically drawn upside down, in the sense that the leaves are at the bottom of the tree.
- The points along the tree where the predictor space is split are referred to as **internal nodes**.

清华大学统计学研究中心

How to Grow A Tree

- The goal is to find boxes $R_1, \ldots, R_M$ that minimize the RSS, given by

$$\sum_{m=1}^{M} \sum_{i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2 .$$

- Unfortunately, it is computationally infeasible to consider every possible partition of the feature space into M boxes.
- Another issue is that we need to specify M beforehand.

清华大学统计学研究中心

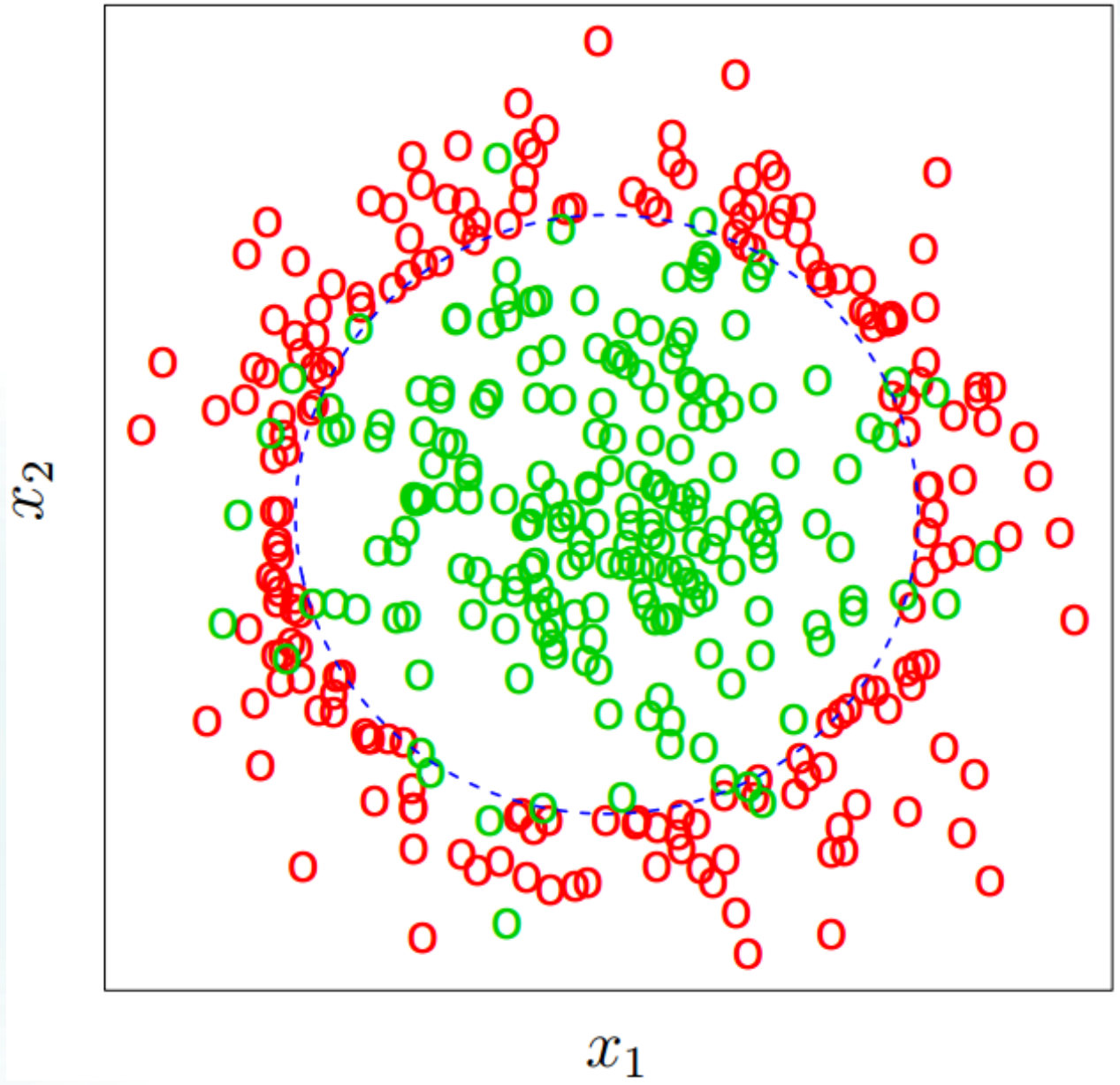
How to Grow A Tree

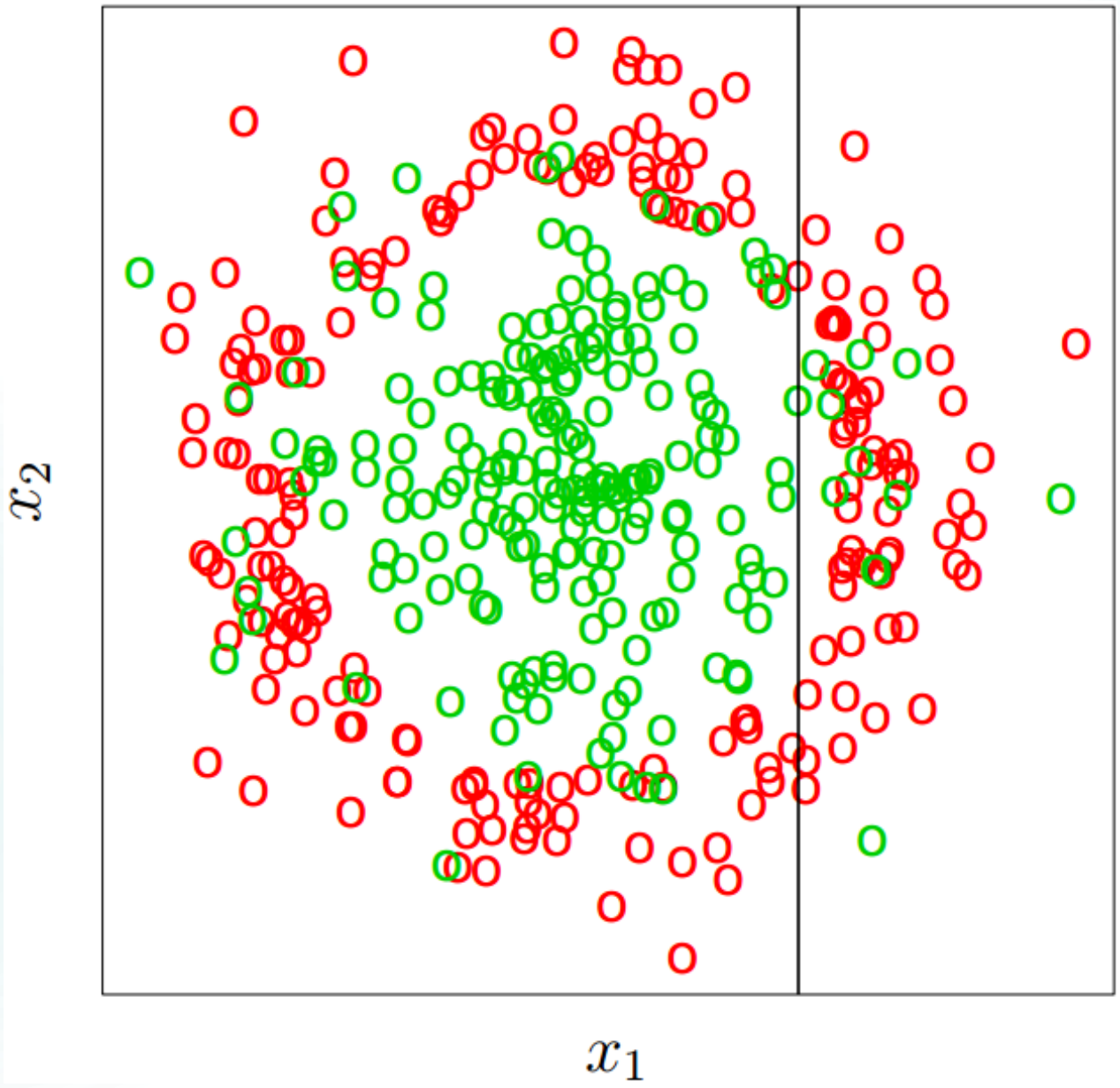
- For this reason, we take a top-down, greedy approach that is known as recursive binary splitting.

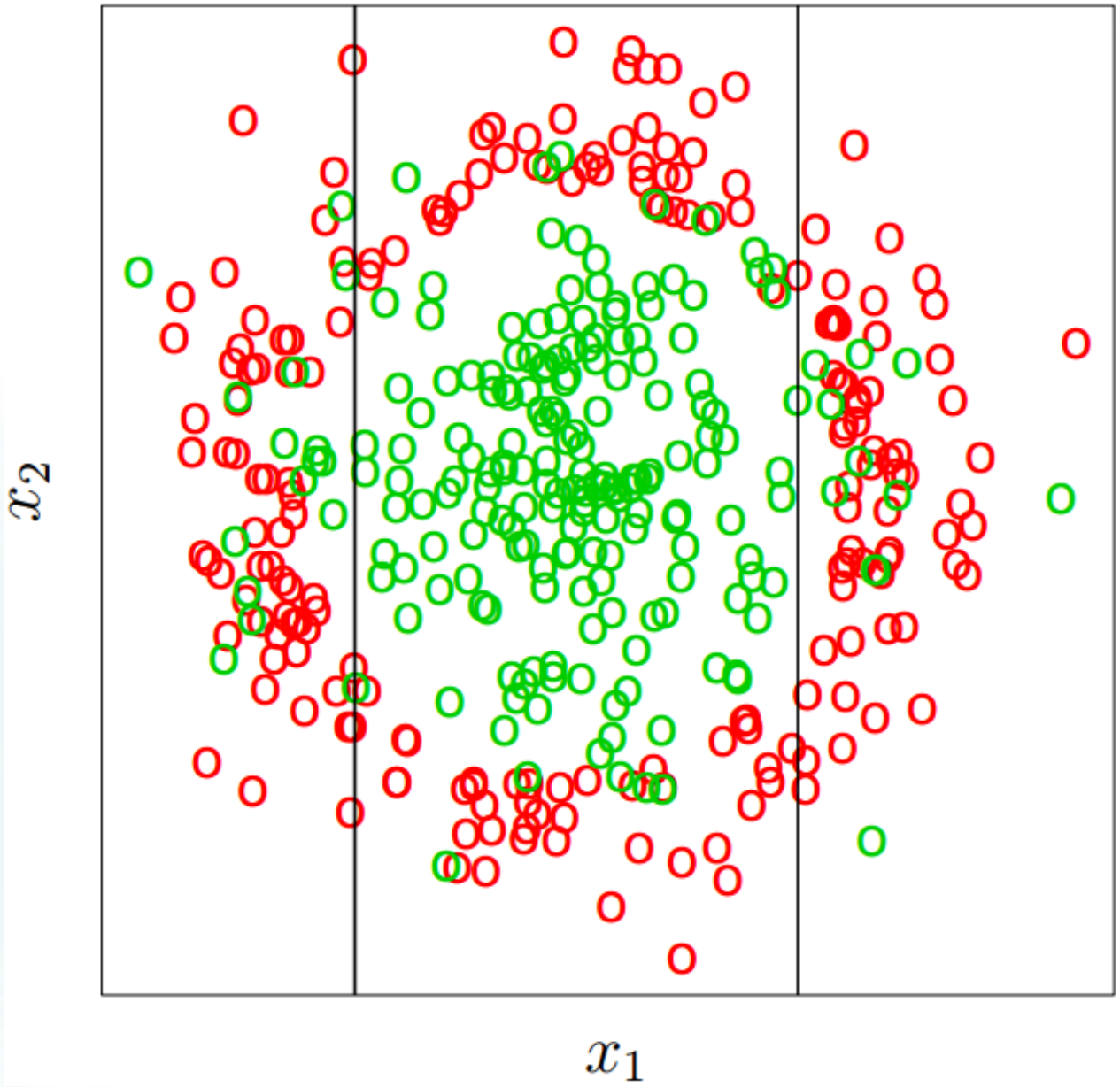
- The approach is **top-down** because it begins at the top of the tree and then **recursively splits** the predictor space; each split is indicated via two new branches further down on the tree.

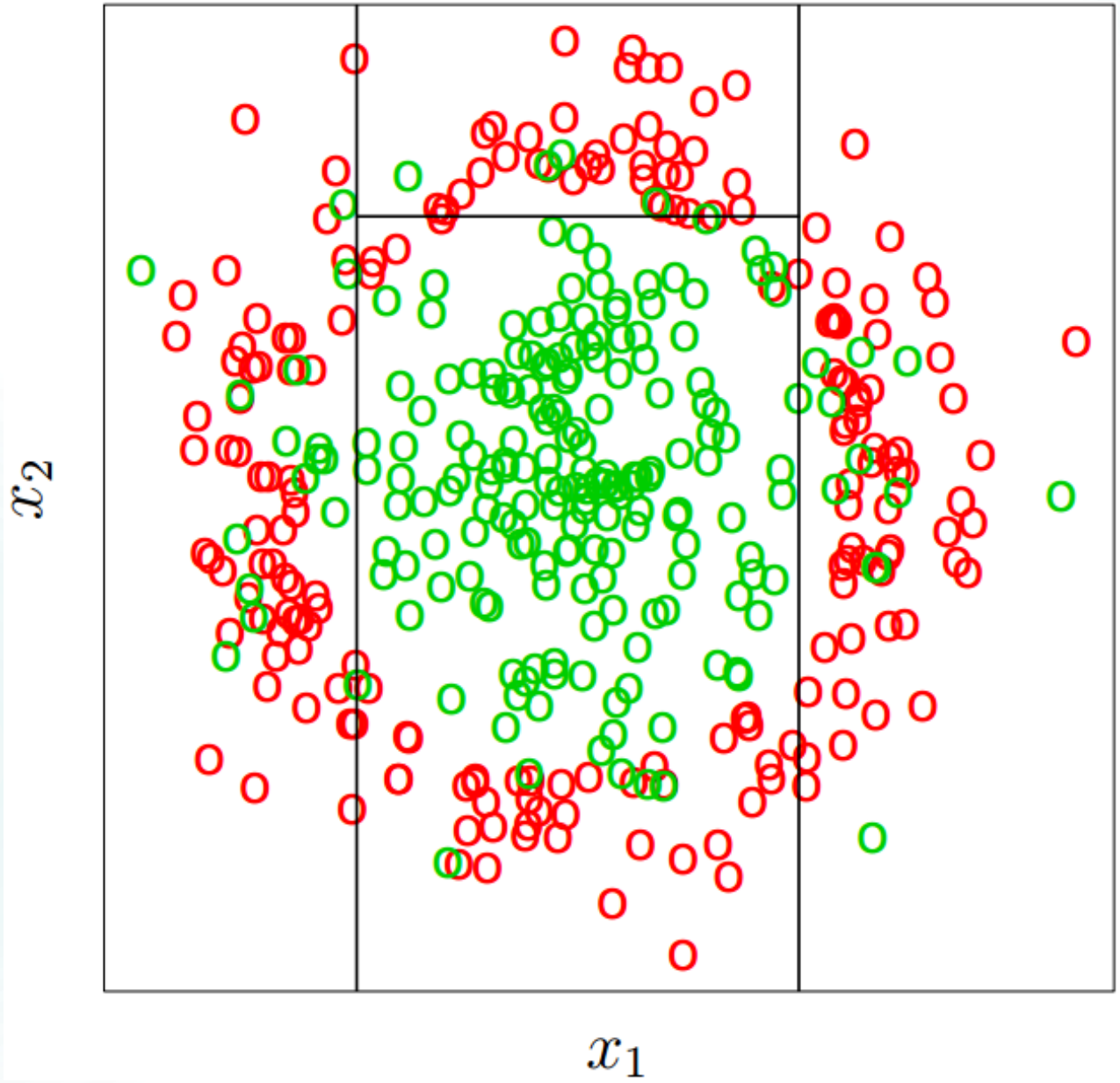
- It is **greedy** because at each step of the tree-building process, the best split is made at that particular step, rather than looking ahead and picking a split that will lead to a better tree in some future step.

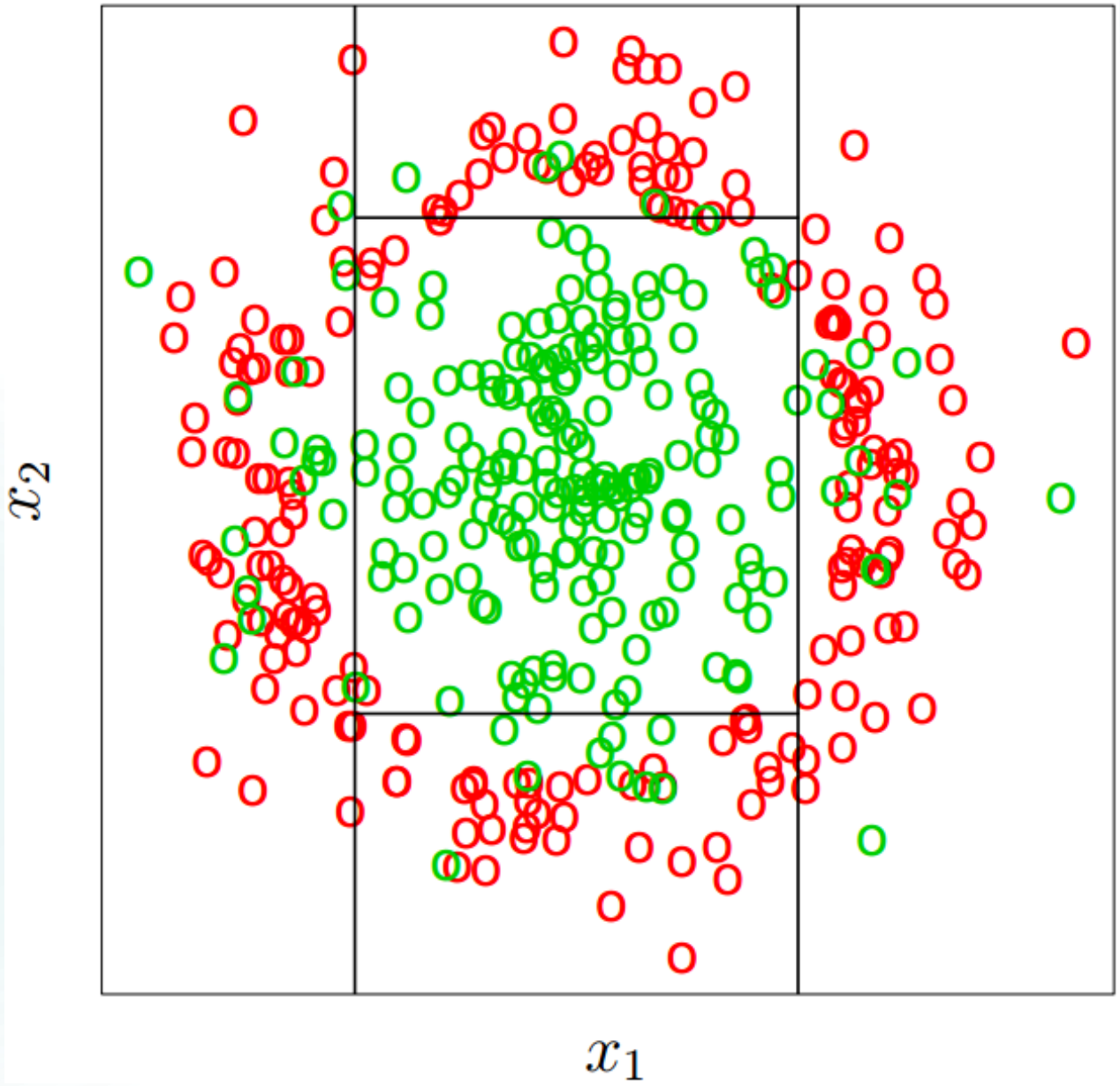
清华大学统计学研究中心

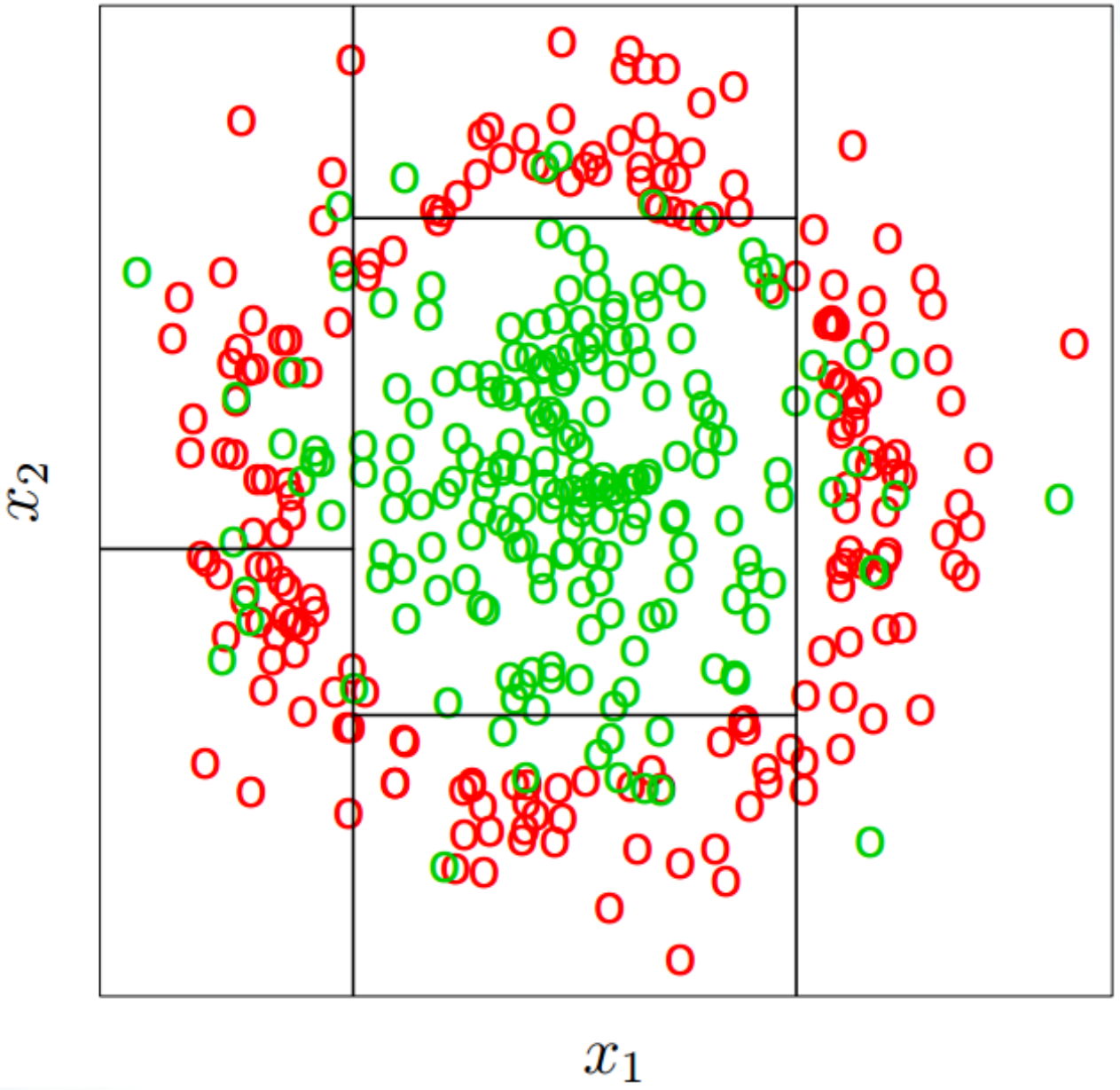

x_2
x_1

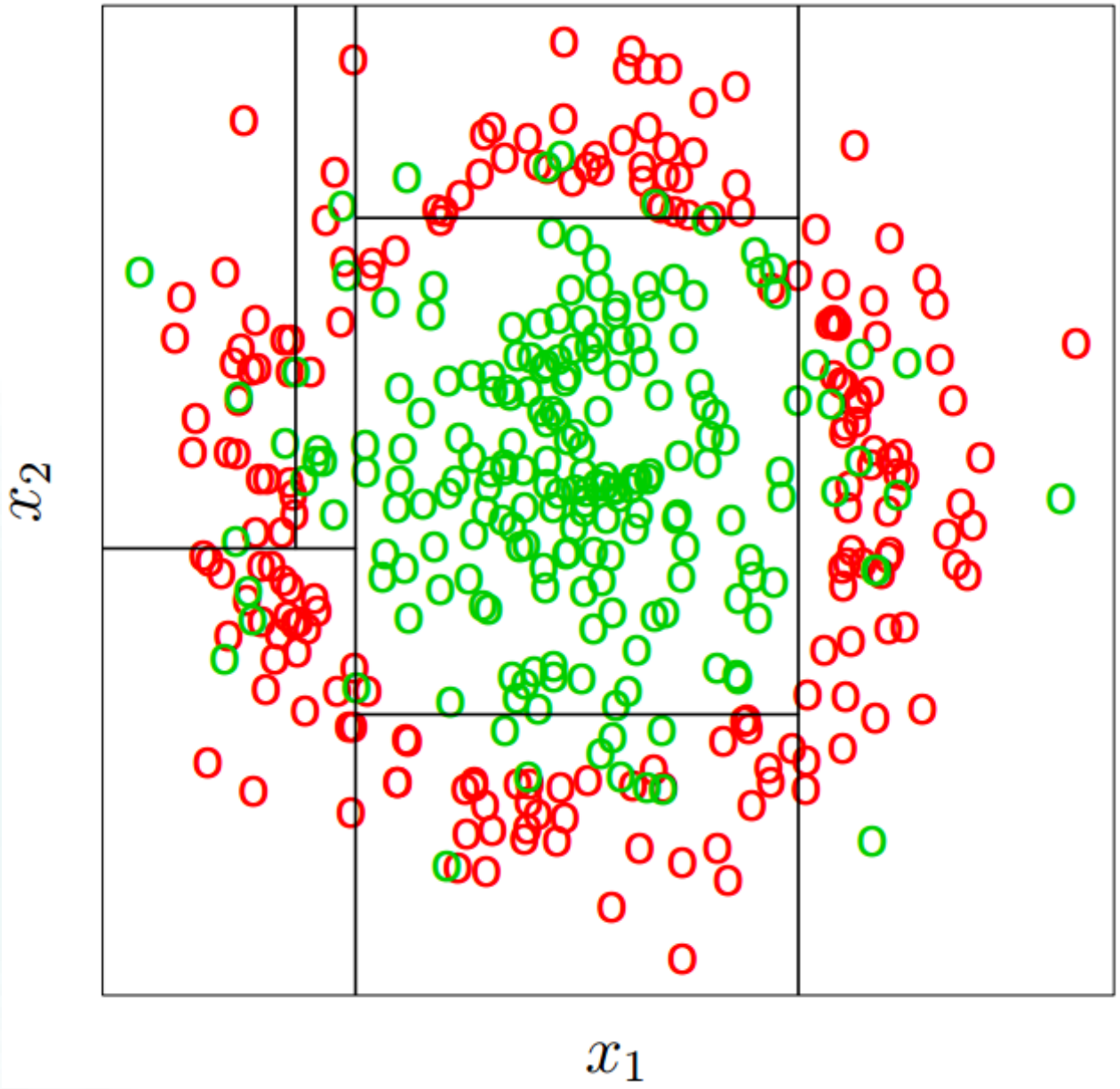

x_2
x_1

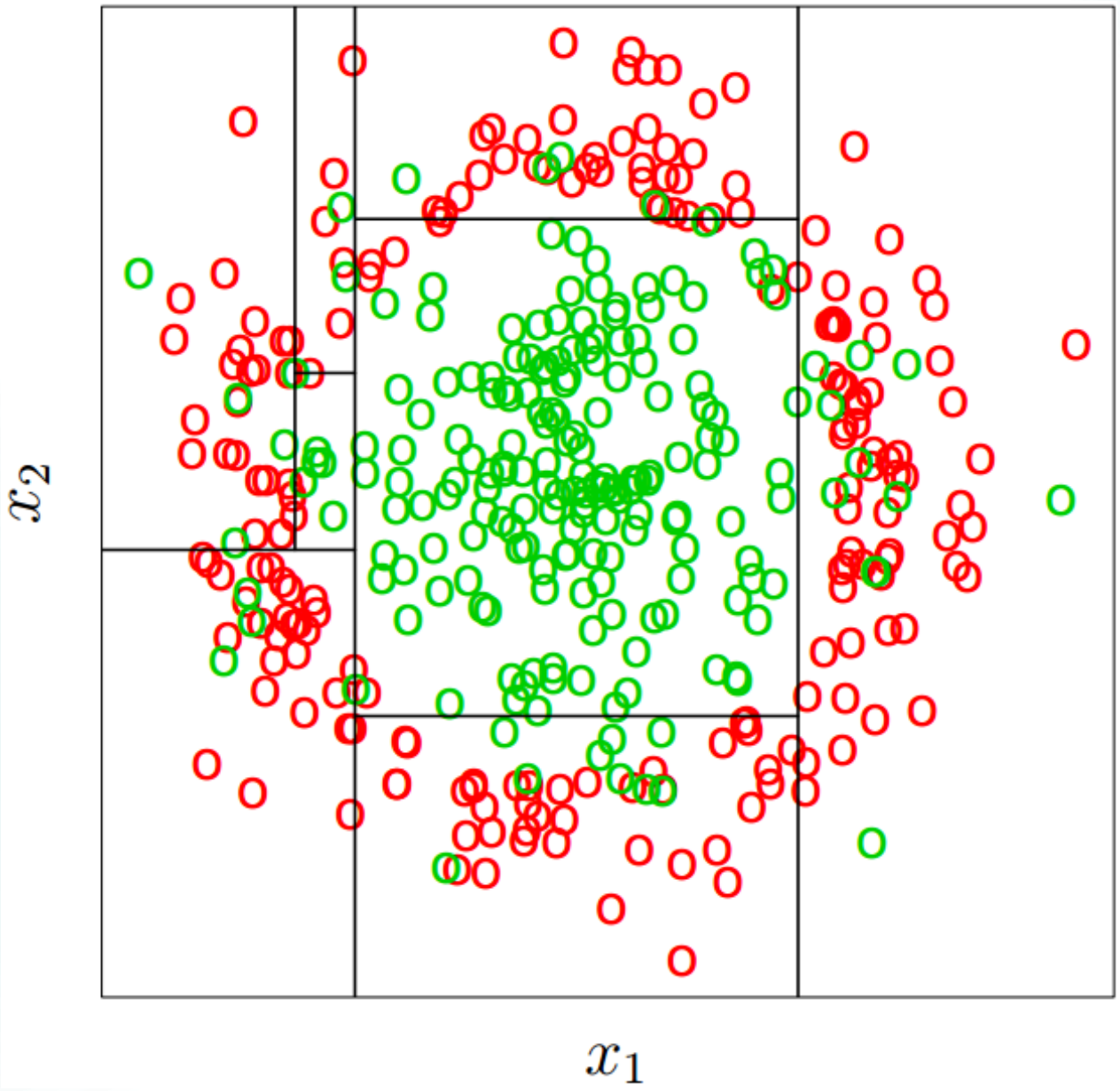

x_2
x_1

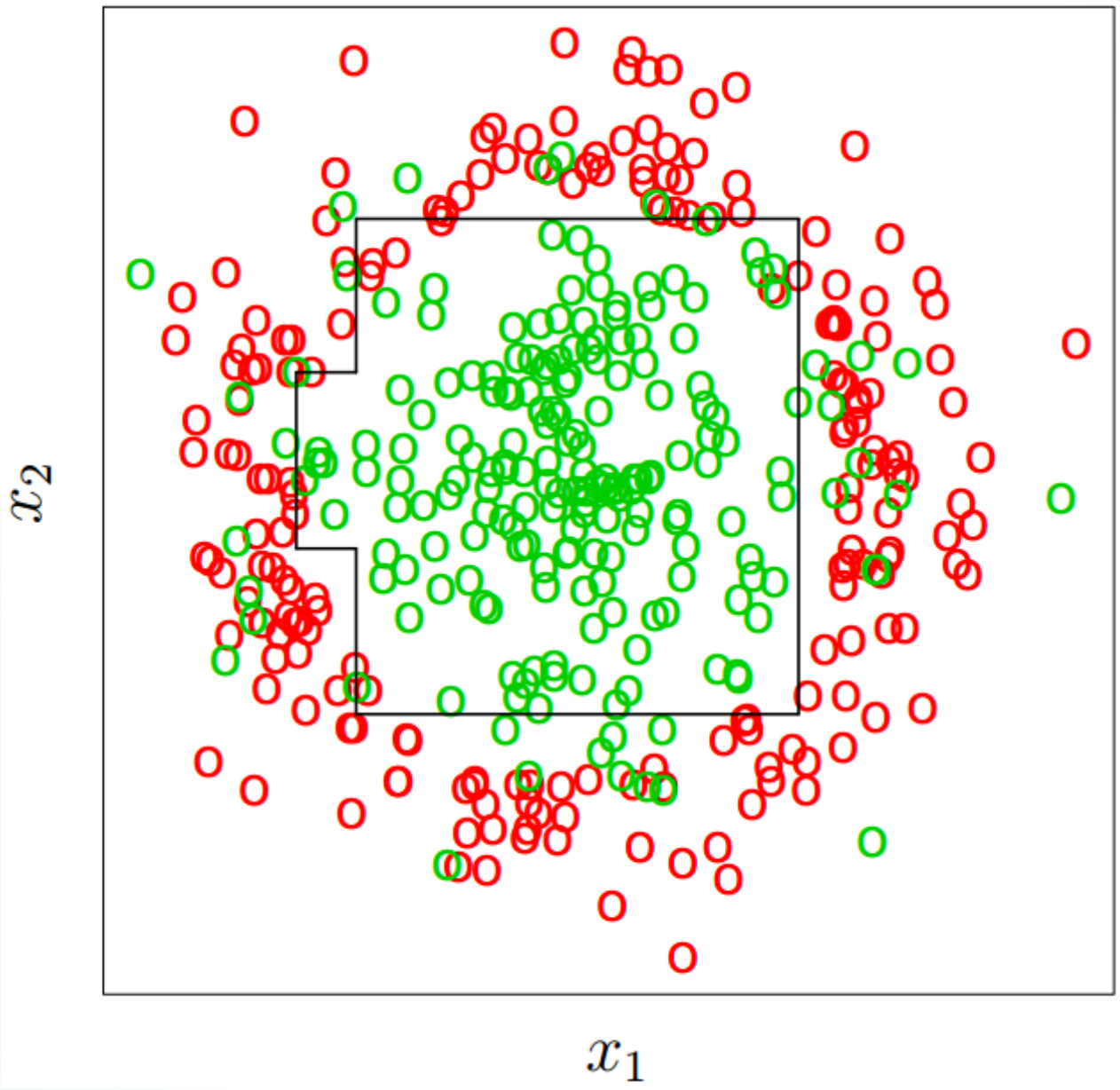

x_2
x_1


x_2
x_1


x_2
x_1


x_2
x_1


x_2
x_1



decision boundary of the decision tree classification

How to Grow A Tree - Details

- Starting with all of the data, consider a splitting variable j and split point s, and define the pair of half-planes

$$R_1(j,s) = \{X|X_j \leq s\} \text{ and } R_2(j,s) = \{X|X_j > s\}.$$

- Then we seek the splitting variable and split point that solve

$$\min_{j,s}\left[\min_{c_1}\sum_{x_i\in R_1(j,s)}(y_i - c_1)^2 + \min_{c_2}\sum_{x_i\in R_2(j,s)}(y_i - c_2)^2\right].$$

- The optimal c_1 and c_2 are obviously the means of the subregions.
- Find the best pair of (j,s) by enumeration.

清华大学统计学研究中心

How to Grow A Tree - Details

- Having found the best split, we partition the data into the two resulting regions and repeat the splitting process on each of the two regions.

- Then this process is repeated on all of the resulting regions, until a stopping criterion, say, a minimum node size (e.g. 5), is reached.

清华大学统计学研究中心

Quiz

- Write down a pseudo-code for the tree-growing procedure (from initiation to termination).
- You need an object model
- and a tree-traversal strategy
- depth first
- breadth first

清华大学统计学研究中心

Pruning A Tree

- The process described above may produce good predictions on the training set, but is likely to *overfit* the data, leading to poor test set performance. *Why?*
- A smaller tree with fewer splits (that is, fewer regions $R_1, \ldots, R_M$) might lead to lower variance and better interpretation at the cost of a little bias.
- One possible alternative to the process described above is to grow the tree only so long as the decrease in the RSS due to each split exceeds some (high) threshold.
- This strategy will result in smaller trees, but is too *short-sighted*: a seemingly worthless split early on in the tree might be followed by a very good split: that is, a split that leads to a large reduction in RSS later on.

清华大学统计学研究中心

Pruning A Tree

- A better strategy is to grow a very large tree T_0, and then prune it back in order to obtain a subtree.
- We define a subtree $T \subset T_0$ to be any tree that can be obtained by pruning T_0, that is, collapsing any number of its internal (non-terminal) nodes. Let $|T|$ denote the number of terminal nodes in T.
- Define the cost complexity criterion

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{x_i \in R_m} (y_i - c_m)^2 + \alpha|T|.$$

- The idea is to find, for each α, the subtree $T_\alpha \subset T_0$ to minimize $C_\alpha(T)$.

清华大学统计学研究中心

Pruning A Tree

- To find T_α we use *weakest link pruning*: we successively collapse the internal node that produces the smallest per-node increase in the RSS, and continue until we produce the single-node (root) tree. This sequence must contain the optimal T_α.
- The optimal α is found by cross-validation.

清华大学统计学研究中心

Algorithm

- 1. Use recursive binary splitting to grow a large tree on the training data, stopping only when each terminal node has fewer than some minimum number of observations.
- 2. Apply cost complexity pruning to the large tree in order to obtain a sequence of best subtrees, as a function of α.
- 3. Use K-fold CV to choose the optimal α. For $k = 1, \ldots, K$:
  - 3.1 Repeat Steps 1 and 2 on the !k fraction of the training data, and get a sequence of subtrees as a function of α.
  - 3.2 Evaluate the error using the left-out part, as a sequence of α. Average the results and find the optimal α.
- 4. Return the subtree from Step 2 that corresponds to the optimal α.

清华大学统计学研究中心

Classification Trees

- Node impurity measures:
  - Misclassification rate:

$$1 - \max_k p_k.$$

  - Gini impurity:

$$\sum_k p_k(1 - p_k).$$

  - Entropy:

$$-\sum_k p_k \log p_k .$$

清华大学统计学研究中心

Gini Impurity

- **Gini impurity** (not to be confused with Gini index) is a measure of how often a randomly chosen element from the set would be incorrectly labeled if it was randomly labeled according to the distribution of labels in the subset.

$$\sum_k p_k(1 - p_k)\,.$$

Entropy

- **Information content** (**self-information**, **surprisal**) I of a random event should be:
  - Nonnegative.
  - The rarer the higher.
  - Additive: If events A and B are independent, $C = A \cap B$, then $I(C) = I(A) + I(B)$.

清华大学统计学研究中心

Entropy

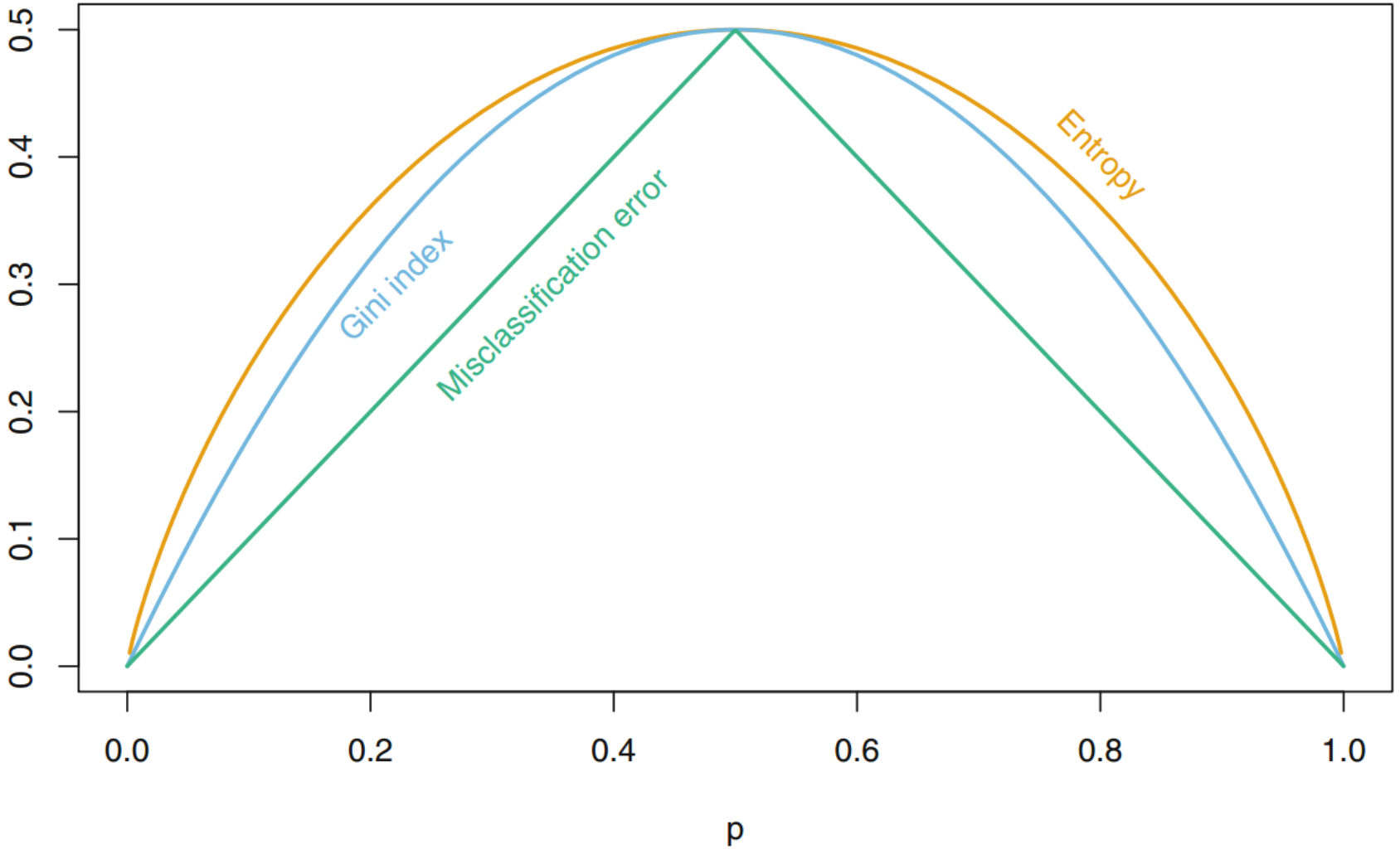
- Therefore,

$$I(X) = \log\left(\frac{1}{P(X)}\right) = -\log P(X).$$

- Entropy: $H(I) = E\{I(X)\} = -\sum_k p_k \log p_k$.

清华大学统计学研究中心

Node Impurity Measures



Splitting Based on Impurity

- When a parent node is split into two children nodes, the quality of the split is computed as

$$Impurity(parent) - \left(\frac{N_{left}}{N_{parent}} Impurity(left) + \frac{N_{right}}{N_{parent}} Impurity(right)\right).$$

- When using entropy as the impurity measure, the above improvement measure is called the "information gain".

清华大学统计学研究中心

Node Impurity Measures

	Class1	Class2	p_1	p_2	Gini	Error
Parent	7	3	7/10	3/10	0.420	3/10
Left	3	0	3/3	0	0	0
Right	4	3	4/7	3/7	0.489	3/7

- $\text{Gini}_{\text{split}} = 0.420 - (3/10 \cdot 0 + 7/10 \cdot 0.489) = 0.078$
- $\text{Error}_{\text{split}} = 3/10 - (3/10 \cdot 0 + 7/10 \cdot 3/7) = 0$
- Gini impurity and entropy encourage pure nodes.

清华大学统计学研究中心

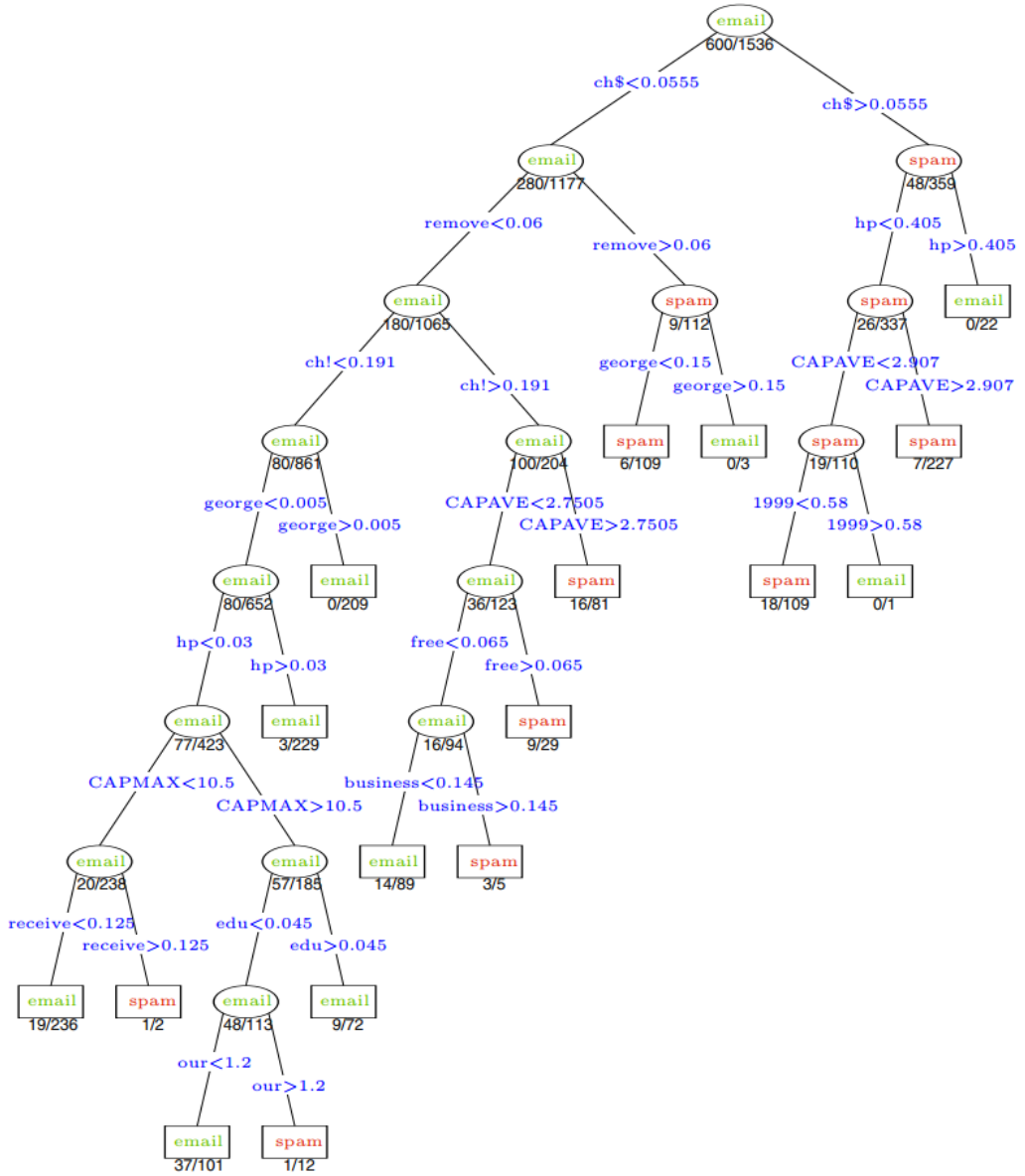
Cost-Complexity Pruning

- For each α, find a subtree T_α that minimizes the cost complexity criterion

$$C_\alpha(T) = \sum_{m=1}^{|T|} N_m \cdot Impurity(R_m) + \alpha|T|.$$

清华大学统计学研究中心

Spam Example



Categorical Variables

- When splitting a predictor having q possible unordered values, there are $2^{q-1} - 1$ possible partitions of the q values into two groups, and the computations become prohibitive for large q.
- However, with a 0-1 outcome, this computation simplifies. We order the predictor classes according to the proportion falling in outcome class 1. Then we split this predictor as if it were an ordered predictor.
- This gives the optimal split in terms of entropy and Gini impurity, as well as for a quantitative outcome and square error loss (the categories are ordered by increasing mean of the outcome).

清华大学统计学研究中心

Advantages of Trees

- Trees are very easy to explain to people. Trees can be displayed graphically, and are easily interpreted even by a non-expert (especially if they are small). In fact, trees are even easier to explain than linear regression!
- Some people believe that decision trees more closely mirror human decision-making than do the regression and classification approaches we've seen earlier.
- Can handle highly nonlinear patterns.
- Trees can easily handle qualitative predictors without the need to create dummy variables.

清华大学统计学研究中心

Disadvantages of Trees

- Limited accuracy when compared to some other regression and classification methods we've covered.
- The tree structure is highly unstable.
- The accuracy can be substantially improved by techniques such as "bagging" and "boosting".

清华大学统计学研究中心

Bagging

Bagging

- **Bagging** (Breiman 1996) is a general-purpose procedure for reducing the variance of a statistical learning method; we introduce it here because it is particularly useful and frequently used in the context of decision trees.
- Recall that given a set of n independent observations $Z_1, \ldots, Z_n$, each with variance σ^2, the variance of the mean $\bar{Z}$ of the observations is given by σ^2/n.
- Similarly, we can average the prediction of decision trees.

清华大学统计学研究中心

Bagging

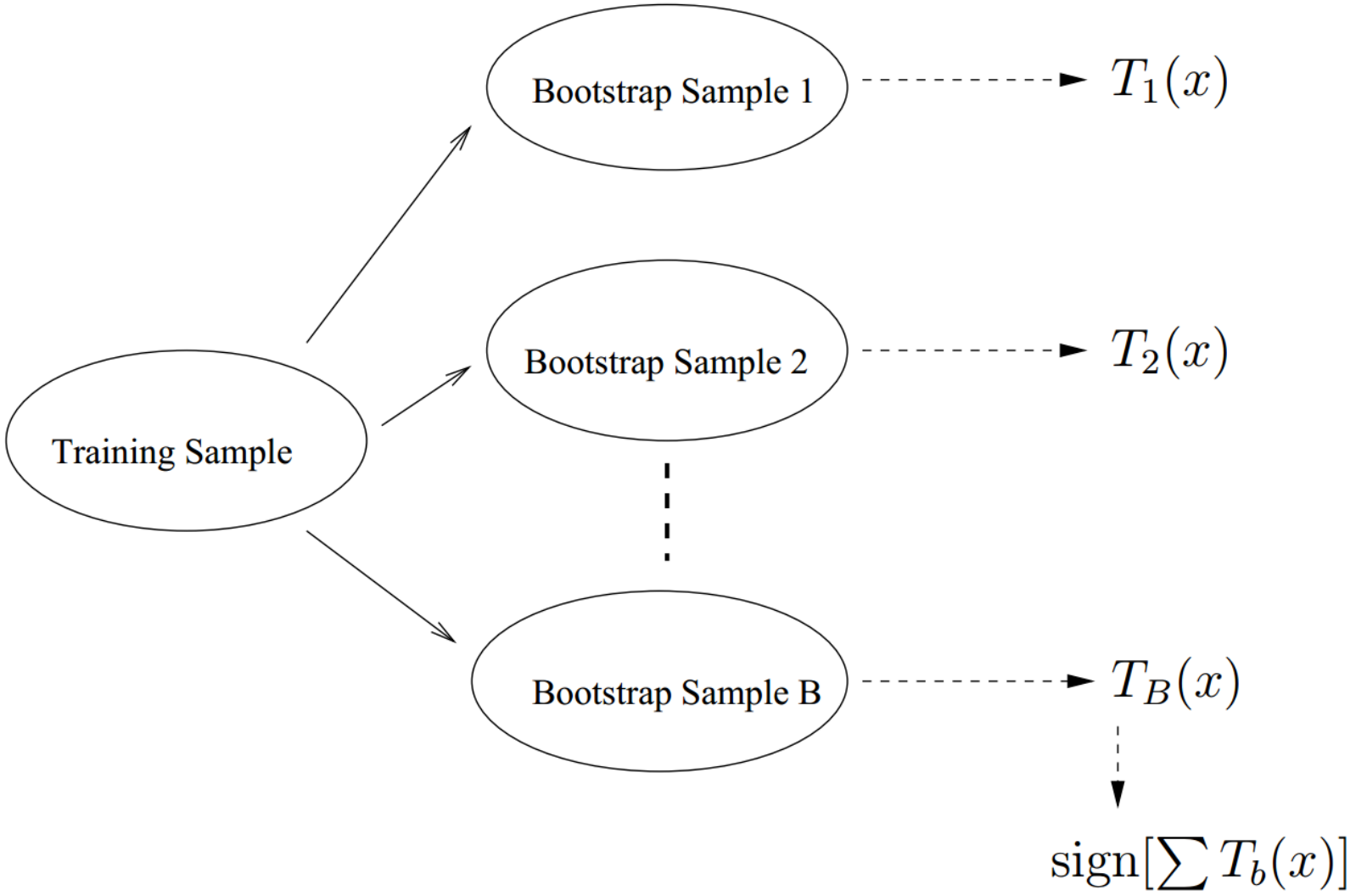
- We generate B different bootstrapped training data sets.
- We then train our model on the bth bootstrapped training set in order to get $\hat{f}_b(x)$, the prediction at a point x.
- We then average all the predictions to obtain

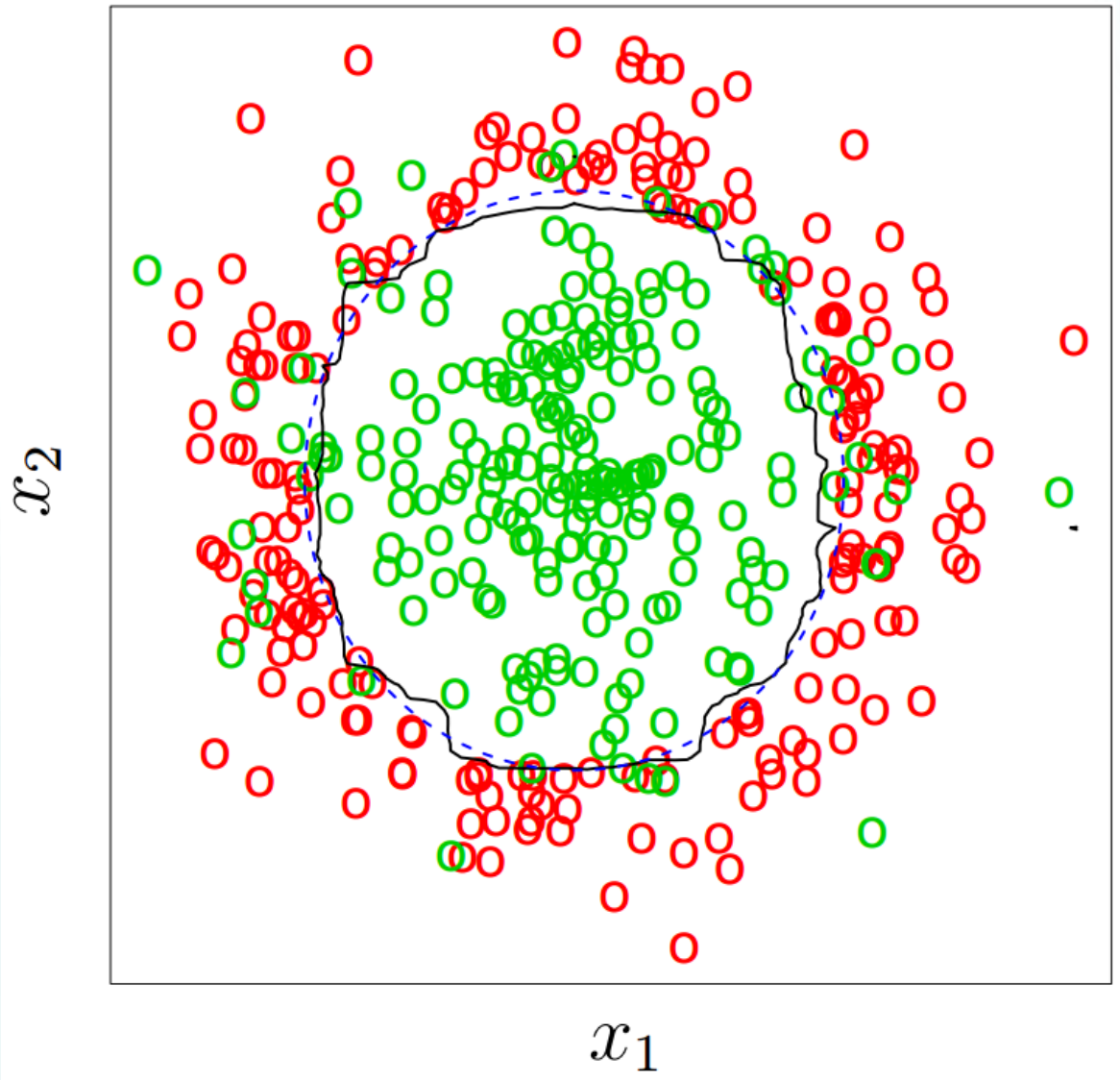
$$\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x).$$

- For classification trees, take a majority vote.

清华大学统计学研究中心

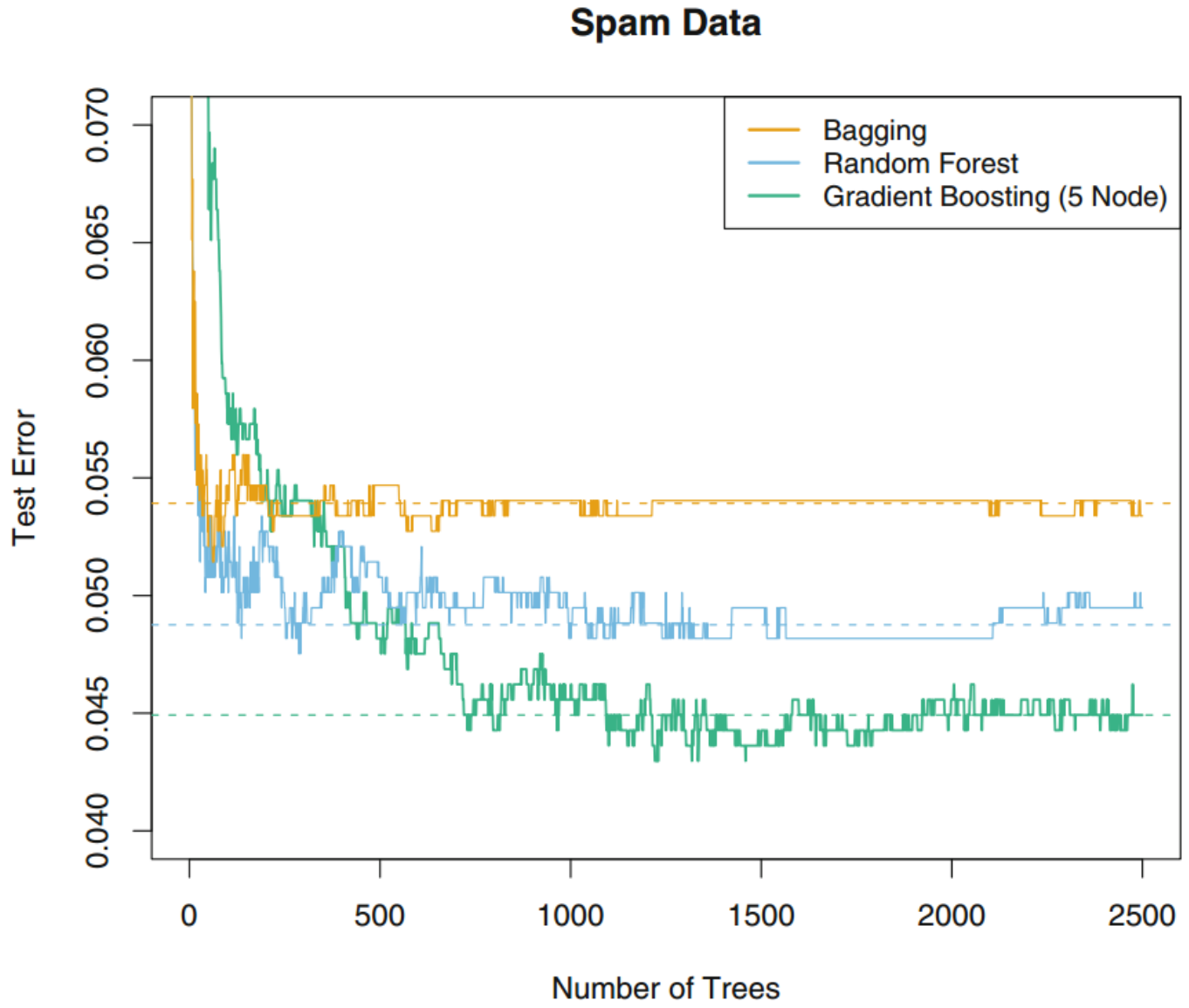
Bagging


Bootstrap Sample 1
$T_1(x)$
Bootstrap Sample 2
$T_2(x)$
Training Sample
Bootstrap Sample B
$T_B(x)$
$\text{sign}[\sum T_b(x)]$



- Bagging averages many trees, and produces more flexible decision boundaries.

Spam Example Cont'd



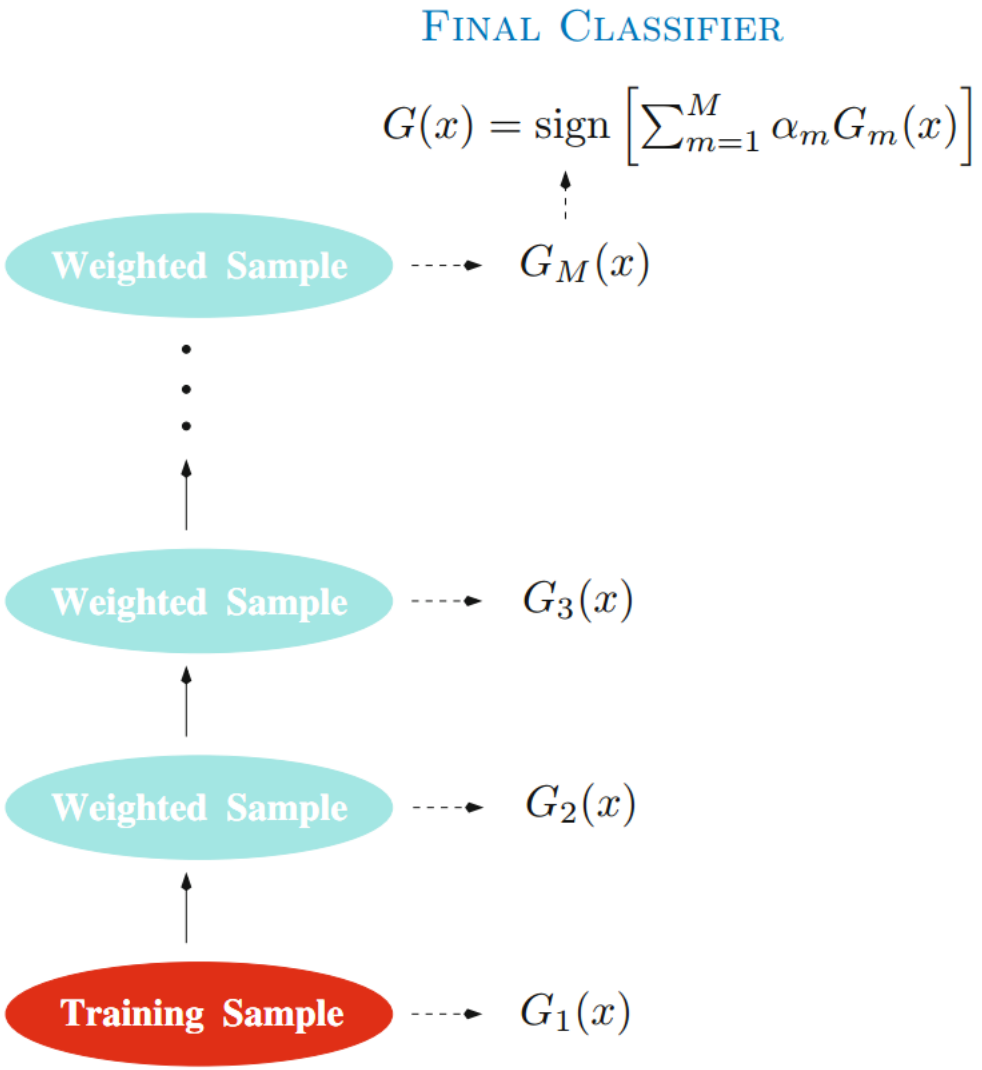
Random Forests

- Random forests (Breiman 1999) provide an improvement over bagged trees by way of a small tweak that decorrelates the trees. This reduces the variance when we average the trees.
- As in bagging, we build a number of decision trees on bootstrapped training samples.
- But when building these decision trees, each time a split in a tree is considered, **a random selection of q predictors is chosen as split candidates** from the full set of p predictors. The split is based on one of the q predictors.
- A fresh selection of q predictors is taken at each split. The inventors suggested $q \approx \sqrt{p}$, but it should better be cross-validated.
- In addition, we generally **do not prune the trees** in random forest. The idea is to allow the trees to overfit, and reduce the variance by averaging.
- One of the best classifiers. Should always be considered a good reference.

清华大学统计学研究中心

Boosting

Boosting


FINAL CLASSIFIER
$G(x) = \text{sign}\left[\sum_{m=1}^{M} \alpha_m G_m(x)\right]$
Weighted Sample
$G_M(x)$
Weighted Sample
$G_3(x)$
Weighted Sample
$G_2(x)$
Training Sample
$G_1(x)$

AdaBoost

- 2003 ACM Gödel Prize: Freund & Schapire
- "… The algorithm demonstrated novel possibilities in analyzing data and is a permanent contribution to science even beyond computer science. Because of a combination of features, including its elegance, the simplicity of its implementation, its wide applicability, and its striking success in reducing errors … the algorithm set off an explosion of research in the fields of statistics, artificial intelligence, experimental machine learning, and data mining…"

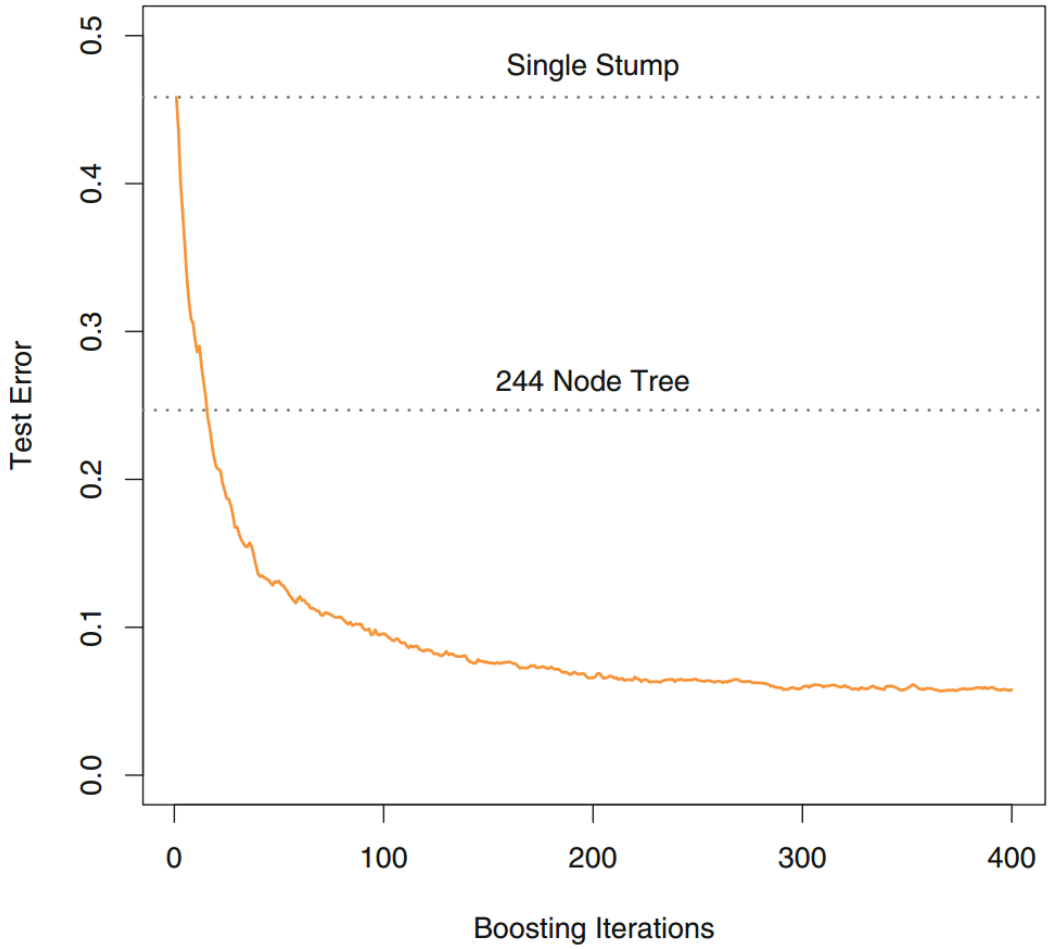
AdaBoost

- In general, Boosting > Bagging > Single Tree.
- "AdaBoost with trees is the best off-the-shelf classifier in the world." — Leo Breiman (1998, also at NIPS 1996).

清华大学统计学研究中心

AdaBoost

- Boosting stumps for classifying spherical patterns. A stump is a two-node tree.



AdaBoost

Algorithm 10.1 *AdaBoost.M1.*

1. Initialize the observation weights $w_i = 1/N,\ i = 1, 2, \ldots, N$.
2. For $m = 1$ to M:
   (a) Fit a classifier $G_m(x)$ to the training data using weights w_i.
   (b) Compute
   $$\text{err}_m = \frac{\sum_{i=1}^{N} w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i}.$$
   (c) Compute $\alpha_m = \log((1 - \text{err}_m)/\text{err}_m)$.
   (d) Set $w_i \leftarrow w_i \cdot \exp[\alpha_m \cdot I(y_i \neq G_m(x_i))],\ i = 1, 2, \ldots, N$.
3. Output $G(x) = \text{sign}\left[\sum_{m=1}^{M} \alpha_m G_m(x)\right]$.

清华大学统计学研究中心

Additive models

- Boosting is a way of fitting an additive expansion in a set of elementary basis functions and takes the form

$$f(x) = \sum_{m=1}^{M} \beta_m b(x; \gamma_m) .$$

- The parameters $\{\beta_m\}$ and $\{\gamma_m\}$ are determined by minimizing the total loss:

$$\min_{\{\beta_m, \gamma_m\}_{m=1}^{M}} \sum_{i=1}^{N} L\left(y_i, \sum_{m=1}^{M} \beta_m b(x; \gamma_m) \right)$$

清华大学统计学研究中心

Additive models

- The previous optimization problem is very hard to solve, due to the large number of parameters, but it can be substantially simplified by "Forward Stagewise Additive Modeling".

Algorithm 10.2 *Forward Stagewise Additive Modeling.*

1. Initialize $f_0(x) = 0$.
2. For $m = 1$ to M:
   (a) Compute

$$(\beta_m, \gamma_m) = \arg\min_{\beta,\gamma} \sum_{i=1}^{N} L(y_i, f_{m-1}(x_i) + \beta b(x_i; \gamma)).$$

   (b) Set $f_m(x) = f_{m-1}(x) + \beta_m b(x; \gamma_m)$.

清华大学统计学研究中心

AdaBoost

- AdaBoost uses the exponential loss:

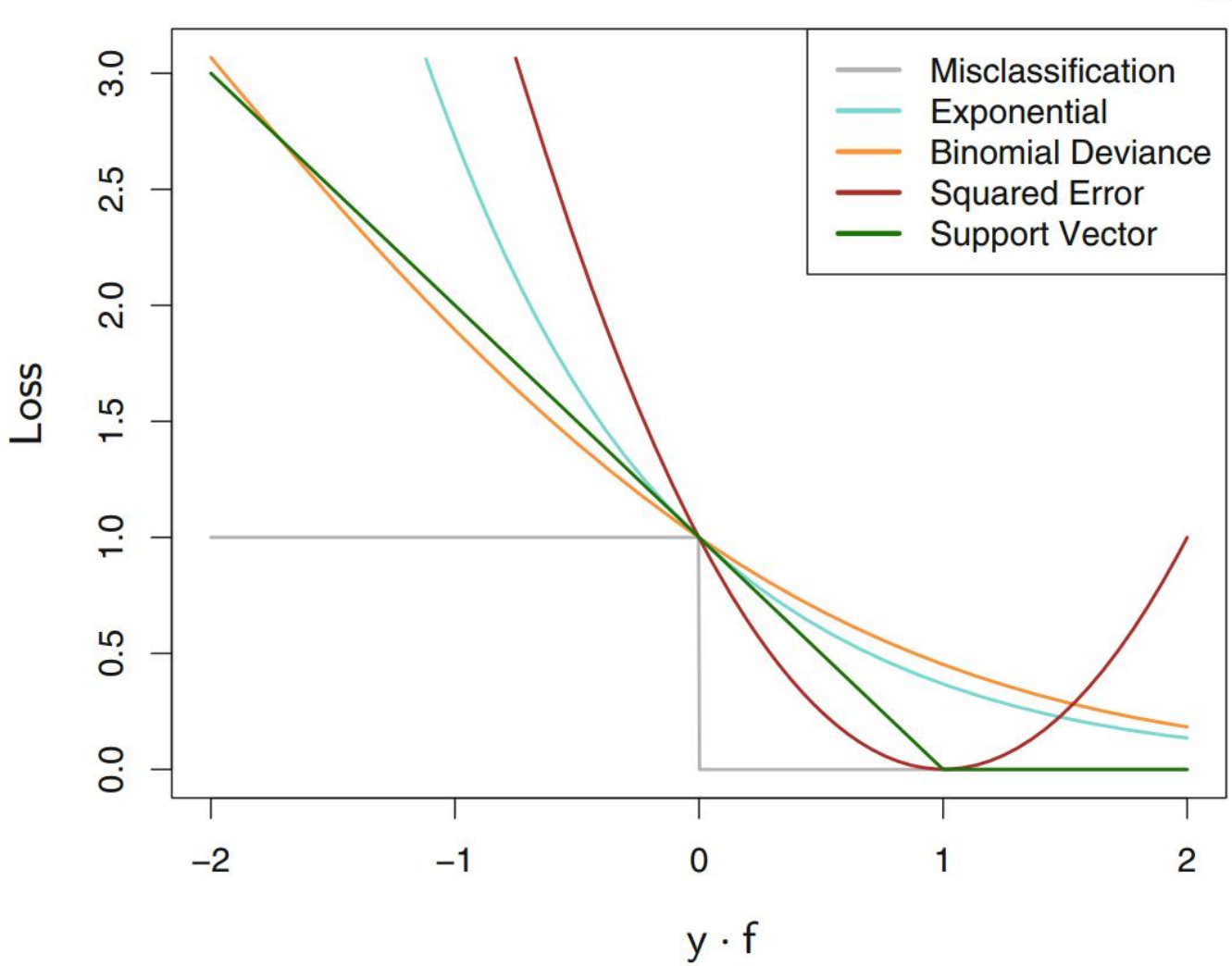
$$L\big(y, f(x)\big) = \exp\big(-yf(x)\big).$$

- For AdaBoost the basis functions are the individual classifiers $G_m(x) \in \{-1,1\}$. Using the exponential loss function, one must solve

$$(\beta_m, G_m) = \underset{\beta, G}{\operatorname{argmin}} \sum_{i=1}^{N} \exp\Big(-y_i\big(f_{m-1}(x_i) + \beta G(x_i)\big)\Big).$$

See Hastie et al. Chapter 10.4 for the derivation of the Adaboost algorithm.

清华大学统计学研究中心



- Larger penalty on wrong classifications than binomial deviance and hinge loss, thus not ideal for noisy training data.

Gradient boosting

Algorithm 10.3 *Gradient Tree Boosting Algorithm.*

1. Initialize $f_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$.
2. For $m = 1$ to M:
   (a) For $i = 1, 2, \ldots, N$ compute
   $$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}.$$
   (b) Fit a regression tree to the targets r_{im} giving terminal regions R_{jm}, $j = 1, 2, \ldots, J_m$.
   (c) For $j = 1, 2, \ldots, J_m$ compute
   $$\gamma_{jm} = \arg\min_\gamma \sum_{x_i \in R_{jm}} L\left(y_i, f_{m-1}(x_i) + \gamma\right).$$
   (d) Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.
3. Output $\hat{f}(x) = f_M(x)$.

清华大学统计学研究中心

Comparison

	AdaBoost	Gradient boosting
Task	Classification	Regression Classification
Basic learner	Classification Trees	Regression Trees
Objective at each step	Minimize the expected loss	Steepest gradient descent of the loss for each sample
Loss function	Exponential loss	Any differentiable loss function

清华大学统计学研究中心

XGBoost

- An “extremely tweaked” implementation of gradient boosting
  - Regularized tree (pruning)
  - Newton-Raphson instead of Steepest Descent
  - Extra randomization of parameters
  - ……